MSC DATA SCIENCE - PORTFOLIO PROJECT

# Review Intelligence Pipeline

A scheduled NLP monitoring system over e-commerce reviews. It detects what customers complain about, per product, per week, and raises an alert when a product degrades or when the model itself drifts.

### The one-sentence pitch

"I built a small monitoring system, like the kind that watches a production ML model in industry, except I built both halves myself: a model that scores review sentiment two different ways, and a detector that watches those scores and the raw review stream for signs something changed."

### Why this is worth talking about, not just the models

- It isn't a notebook that trains one model and stops. It's a pipeline with seven stages, each reading and writing real files, that could run on a schedule.
- I found and fixed two real bugs during development that weren't in any tutorial: a train/serve data leakage bug, and a false-positive alert bug. Both are documented with the exact numbers before and after the fix.
- Every non-obvious decision - why this threshold, why this window size, why this library default was wrong - is written down with the trade-off I considered, not just the choice I made.
- The detection logic itself (pipeline/detect.py) is code I wrote by hand, not generated for me, specifically so I can defend it from first principles.

| No paid APIs | CPU only | 7 stages | Hand-written |
| --- | --- | --- | --- |
| everything runs locally | 8GB RAM assumed | ingest to detect | detection logic |

## 01 How the pipeline is put together

The pipeline is not one straight line. ingest, validate, and score run in sequence, but aggregate branches directly off validate and never waits for scoring, embedding, or topic modeling to finish. detect is the only stage that reads from two different places at once: the weekly stats table and the raw scored rows.

### 1. ingest

Reads: Raw Amazon reviews (local file)
Writes: Date-partitioned Parquet, overwrite-by-partition

Rolling window widened 30 -> 200 -> 730 days over the project's life.

### 2. validate

Reads: Ingested Parquet
Writes: Valid rows forward; failures to a quarantine table with a reason code

Nothing is silently dropped - every rejected row is inspectable later.

### 3. score

Reads: Validated Parquet
Writes: Two sentiment scores per review, side by side

A fast TF-IDF baseline and a slower DistilBERT model score every row independently, so they can be compared.

### 4. embed

Reads: Scored rows, negative only (rating<=2)
Writes: Sentence embeddings (MiniLM)

Filters by the reviewer's own star rating, never a model's prediction - nothing here can leak.

### 5. topics

Reads: Embeddings
Writes: A topic id per negative review, via transform() only

The topic model is never refit automatically - refitting silently changes what every topic id means.

### 6. aggregate

Reads: Validated Parquet, via DuckDB
Writes: Review count + mean rating per product per week

The table pipeline/detect.py actually watches for problems.

### 7. detect

Reads: Weekly stats + scored rows
Writes: Alerts (SQLite) + a drift number (PSI) per run

Hand-written by me, not scaffolded - the part of the project I can defend line by line.

## 02 Decisions worth explaining out loud

Full reasoning for every entry below lives in docs/decisions.md, with the actual data checked at the time. These are the ones most worth walking through in a conversation.

### Why the model is never trained on data it will later score

The baseline model is only ever trained on reviews strictly before the detection window starts. I actually broke this once: when I widened the ingest window from 30 to 200 days without re-checking, the model ended up scoring 169 days of data it had already seen during training. I found it by checking the date boundaries by hand, not by assuming the config change was safe, and fixed it by widening further to 2 years and retraining against the new boundary.

### Why two different thresholds for two different alert checks

detect.py flags a product's week as anomalous using a z-score: how many standard deviations a value sits from that product's own history. At first I used the same threshold (2.0) for both the review-count check and the rating check, tuned for how short these per-product histories were early on. Once the rating check alone was producing 259 of 313 total alerts - the large majority, with no secondary condition backing it up - I tightened it to 4.0, which dropped it to 12 genuinely extreme cases while leaving the count check untouched. Two different thresholds for two different signals, chosen deliberately, not inherited.

### Why stop-word removal was turned off for the sentiment model

scikit-learn's default English stop-word list strips words like "not" and "no". For sentiment analysis, that silently destroys negation - "not good" and "good" would look identical to the model. Turning stop-word removal off was the right call specifically because of what the task is.

### Why every model has a pinned version, never "whatever trained last"

Every trained model is content-addressed with a sha256 hash, and every pretrained model (DistilBERT, MiniLM) is pinned to an exact Hugging Face git revision. Nothing is picked up automatically - bumping the active model is a deliberate, visible edit to a config file, so a scoring run never silently changes behavior underneath you.

## 03 Real bugs I found and fixed

These matter more than the model numbers, because they show the habit of checking real output rather than trusting that code which runs without erroring is correct.

### 1. Train/serve data leakage (the window-widening bug)

Widening the ingest window from 30 to 200 days, done for a different reason (more weekly history per product), accidentally created 169 days of overlap between the baseline's training data and what it was now scoring. Found by checking the actual date boundaries, not assumed safe. Fixed by widening to 2 years and retraining against the new boundary.

### 2. A product with zero bad reviews fired a complaint alert

Product B07D5ZGJF1 had a perfect 5.0 average rating every single week, yet triggered a complaint_volume_spike alert. The bug: the volume check only looked at whether review count spiked relative to a product's own (often tiny) baseline - it never checked whether those extra reviews were actually negative. Fixed by requiring that week's rating to also sit below the product's own average. complaint_volume_spike alerts dropped from 190 to 54 as a direct result - most of the earlier ones were this exact false-positive pattern.

### 3. A stale-alerts gap, found twice, independently

The alerts table is written with INSERT OR IGNORE, keyed on (product, week, alert type). That only prevents an exact duplicate - it has no way to remove an alert that becomes invalid once the detection logic or window changes. This surfaced twice: once after the window widened (27 stale alerts left behind), and again after the complaint-volume fix above (136 stale alerts left behind). Both times I found it by comparing the printed count to the table's actual row count, not by assuming they matched. It's documented as a known, unfixed gap rather than quietly patched around - a 'what would you do differently' answer ready for an interview.

## 04 The real numbers

All numbers below are from real runs against the current 2-year window, not estimated or illustrative.

### Sentiment baseline vs. DistilBERT

| 0.973 / 0.728 | 90.8% | 34,497 vs 24 |
|---|---|---|
| baseline F1, positive / negative class | baseline <-> DistilBERT agreement | rows/sec, baseline vs DistilBERT |

The accuracy headline (0.951) looks strong mainly because the data is ~92.8% positive reviews. The honest story is per-class: negative-class recall is 0.914 (catches 91% of real complaints) but precision is only 0.605 (2 in 5 reviews called "negative" are actually positive). That's the direct, expected cost of training with class_weight="balanced" instead of undersampling - and it's the right trade for a monitoring system, where missing a real complaint costs more than one extra false alarm.

### Topic model

47 topics found; 41.5% of training reviews (3,270 of 7,874) landed in the outlier bucket rather than any cluster. Fragmentation is a real, recurring pattern: topics for "cracked CD case" and for Beatles-bootleg warnings each split into 2-3 near-duplicate clusters, surviving a full refit. Whether that needs retuning hdbscan's parameters is still an open question.

### Detection

| 969 | 54 | 12 |
|---|---|---|
| eligible products (>= 3 weeks history) | complaint-volume-spike alerts | rating-drop alerts |

Model drift (PSI, first-half vs. second-half of all 15,003 scored reviews): baseline 0.0042, DistilBERT 0.0032 - both comfortably under the 0.1 'no significant shift' rule of thumb, as expected with no real drift injected into this static historical dataset.

### Drift experiment (synthetic)

60 simulated days; from day 30, reviews from an unrelated category (Gift_Cards) replace the normal stream entirely. Both models' PSI rises sharply at the injection point (baseline 0.041 -> 0.076 day-over-day; DistilBERT 0.210 -> 0.551) - proof the drift check actually reacts to a real distribution shift, not just noise.

## 05 What I'd flag before anyone else finds it

A project that only shows strengths invites exactly the questions it can't answer. These are the real, current gaps - stated plainly rather than hidden.

- The alerts table doesn't reconcile itself. insert-or-ignore only blocks exact duplicates; it never removes an alert that a later config or logic change invalidates. Documented, hit twice in practice, not yet fixed in main().
- 41.5% of negative reviews land in the topic model's outlier bucket. I haven't yet determined whether that's an acceptable property of this category's data or a sign hdbscan needs retuning.
- I haven't done a systematic look at the 1,386 rows (9.2%) where the baseline and DistilBERT disagree - I have one hand-picked example, not a pattern across all of them.
- DistilBERT's PSI is noticeably noisier than the baseline's even before any real drift is injected (mean 0.144 vs 0.064 pre-injection). I don't yet know if that means it needs a larger reference window or is just an inherently noisier signal on this data.
- CI's scheduled run depends on GitHub Actions' cache for trained model artifacts, since nothing is committed to git and there's no paid storage. A cache eviction fails the run loudly - a known trade-off of the no-paid-services constraint, not an oversight.

### Bottom line

This is in a genuinely presentable state: the strengths are real engineering judgment (leakage prevention, deliberate versioning, honest metric framing), not polish. The open items above are exactly the kind of thing worth saying out loud first, framed as 'here's what I'd tackle next' - that reads as self-awareness, not as an unfinished project.

## 06 How this was actually built

Worth saying plainly, because it's the kind of question that comes up: I used Claude Code as a collaborator for scaffolding, not as a replacement for understanding the project.

- Claude Code built the infrastructure for each of the 7 pipeline stages, the test suite, and the habit (enforced by a project-level instruction file) of writing down every non-obvious decision as it was made.
- I wrote pipeline/detect.py - the actual detection logic - by hand, line by line, deliberately kept out of Claude's scope. That's the part of the project most likely to get interview follow-up questions, so it's the part I can actually defend from first principles.
- The evaluation harness (evaluation/) follows the same rule - the harness scaffolding is shared, but the logic and interpretation are mine.
- Every real bug in this document (the leakage bug, the false-positive alert, the stale-alerts gap) was found by checking actual output against what I expected, not by assuming code that ran without errors was correct.

### A 60-second version, if someone only has a minute

"It's a small ML monitoring pipeline over product reviews - think of it as a miniature version of what a data team would run in production to catch when a product starts getting worse reviews, or when their sentiment model starts drifting. I built it in seven stages, found and fixed two real bugs along the way that a tutorial wouldn't have warned me about, and wrote the actual anomaly-detection logic myself so I can defend every threshold choice in it, not just the parts that were scaffolded for me."